Daniel Amissah
February 25, 2026
CMPE310
Lab 2 Report

Overview:

This lab focuses on calculating the Hamming distance between two user-entered strings. The Hamming distance is the number of bit positions that differ when comparing the ASCII values of each character. The program uses C to handle input and an assembly routine to perform the bit-level comparison.

C File Algorithm:

a. Prompt the user for the first and second strings.
b. Measure the length of both strings.
c. Remove newline characters left by fgets.
d. Determine the shorter of the two lengths.
e. Call the assembly function with both strings and the shorter length

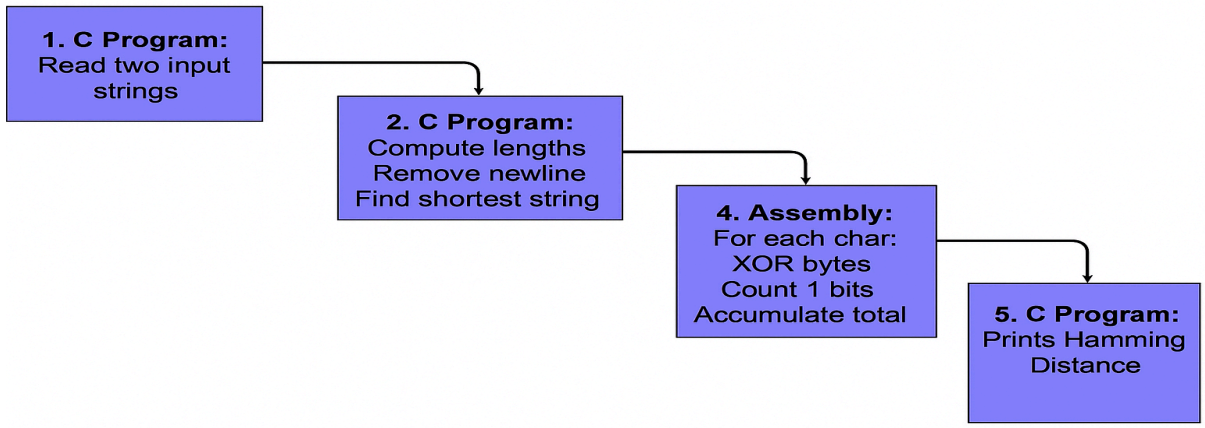
Assembly File Algorithm:

f. Load one byte at a time from each string.
g. XOR the two bytes to identify differing bits.
h. Count the number of 1 bits in the XOR result.
i. Repeat this for the number of characters equal to the shorter string.
j. Return the total count in RAX.

Why is this the most Optimal:

This method is optimal because XOR directly reveals bit differences, which is exactly what the Hamming distance measures. It also uses simple arithmetic and logic instructions that we covered in lecture, making it efficient and aligned with low-level operations.

Flowchart:



Daniel Amissah
February 25, 2026
CMPE310
Lab 2 Report

Code:

Gas:

```
.section .bss
.lcomm buf1, 256        # buffer for string 1
.lcomm buf2, 256        # buffer for string 2

.section .text
.global hamming_distance

hamming_distance:
    # rdi = pointer to string 1
    # rsi = pointer to string 2
    # rdx = length of shorter string

    xor %rax, %rax      # total hamming distance = 0
    xor %rcx, %rcx      # loop counter = 0

char_loop:
    cmp %rcx, %rdx      # if counter == length, we are done
    je done

    mov (%rdi, %rcx, 1), %bl   # grab byte from string 1
    xor (%rsi, %rcx, 1), %bl   # XOR with byte from string 2

    mov $8, %r8b        # check 8 bits
loop:
    test $1, %bl
    jz skip
    inc %rax
skip:
    shr $1, %bl
    dec %r8b
    jnz loop

    inc %rcx
    jmp char_loop

done:
    ret
```

Daniel Amissah
February 25, 2026
CMPE310
Lab 2 Report

C:

```
#include <stdio.h>
#include <string.h>

extern int hamming_distance(char *s1, char *s2, int len);

int main() {
    char str1[256], str2[256];

    printf("Enter string 1: ");
    fgets(str1, 256, stdin);
    str1[strcspn(str1, "\n")] = 0;

    printf("Enter string 2: ");
    fgets(str2, 256, stdin);
    str2[strcspn(str2, "\n")] = 0;

    int len1 = strlen(str1);
    int len2 = strlen(str2);
    int shorter = len1 < len2 ? len1 : len2;

    int result = hamming_distance(str1, str2, shorter);

    printf("Hamming distance: %d\n", result);
    return 0;
}
```

Compiler Command: gcc -no-pie hamming_distance.s hamming.c -o task2

Run Command: ./task2

Sample Outputs:

```
@damissah091-crypto ➜.../codespaces-blank/CMPE 310/CMPE310 Lab/Lab2 $ ./task2
Enter string 1: foo
Enter string 2: bar
Hamming distance: 8
@damissah091-crypto ➜.../codespaces-blank/CMPE 310/CMPE310 Lab/Lab2 $ ./task2
Enter string 1: this is a test
Enter string 2: of the emergency broadcast
Hamming distance: 38
```

Daniel Amissah
February 25, 2026
CMPE310
Lab 2 Report